LiDAR-Based Factor Graph SLAM

Arihant Jain
Electrical and Computer Engineering
UC San Diego

***Abstract*—In the modern day, we have available to us a greater number of sensors than ever before. We can take advantage of these sensors to build more detailed and accurate maps of the world around us. In this report, we will use a LiDAR on a differential-drive setup to perform odometry and occupancy mapping, and add loop closure to improve estimates. We also use a Kinect stereo camera to create a texture map by mapping floor points to pixels.**

***Index Terms*—IMU, LiDAR, Stereo camera, Mapping, Odometry, Factor Graph SLAM, ICP, Texture mapping, Loop closure**

I. Introduction

In SLAM problems, having more sensors is almost always better than having fewer ones. More sensors enables more accurate pose estimation using more complex filtering algorithms. An added advantage to having more sensors is that having sensors that show a representation of the world, such as LiDARs or cameras, enable scan matching and loop closure to better estimate world position and orientation.

There exist many algorithms for accurate mapping and SLAM problems with the sensors we are using, and through this project, we will explore a few of these methods to develop an accurate map of our environment, starting with simpler methods and moving on to more complex ones.

In this project, we build upon IMU odometry by using a differential-drive setup with a LiDAR and a Kinect depth camera, and take advantage of the sensor data to improve our mapping/odometry results. Our goal is to use data from all these sensors to iteratively improve our mapping, starting with IMU odometry, moving onto iterative closest position (ICP) using LiDAR data, followed by texture mapping to build a color map of the world, and finally using Factor Graph SLAM with loop closure to build the most accurate world map.

We will be displaying our results by using 2 datasets, both mapping different areas of a certain building in UCSD. Using the methods described, we will try to build an accurate map of the floor that was used to collect the data. We will discuss our results and their accuracy.

II. Problem Formulation

We split our problem up into multiple tasks: we start with using the wheel encoder to find velocity values, then use the IMU to estimate angular velocity. Next, we use these pose estimates along with LiDAR scans to find the ICP pose estimate. Then, we create a texture map using the depth camera data. Finally, we use gtsam to develop an accurate map using Factor Graph SLAM.

To get distance travelled by each side of wheels in the robot frame using the encoder, we use differential-drive algorithms, taking advantage of consistent motor encoder and IMU data. We then find our IMU angular velocities by only considering the w_z, since the robot is moving in a 2-D plane:

$$w = (w_z)_{imu}$$

We use timestamp matching to find the encoder movement that aligns with the w, and then use the change in time between the timestamps to calculate the world pose of the robot at each timestep.

$$X(t) = X(t-1) + dt * [v_x(t), v_y(t), w(t)]$$

We then use these pose estimates along with LiDAR scans to execute the ICP algorithm to find a more accurate estimate of relative poses, ${}_{t-1}T_t$:

$${}_{t-1}T_t = ICP(scan_{t-1}, scan_t, ({}_{t-1}T_t)_{est})$$
where $({}_{t-1}T_t)_{est} = rp_to_pose(X(t) - X(t-1))$

We can then make an occupancy and texture map, using each LiDAR scan to find the boundaries of the map that we are navigating, and using the Kinect camera and depth data to find the color of the ground at each timestep. We use a log-odds model for occupancy mapping.

Finally for loop closure, we build a factor graph containing the state at each timestep, and use loop closure by proximity, and over a fixed interval. We then optimized our graph and recalculated relative poses using ICP for the loop closures, and updated our motion model and occupancy and texture mapping.

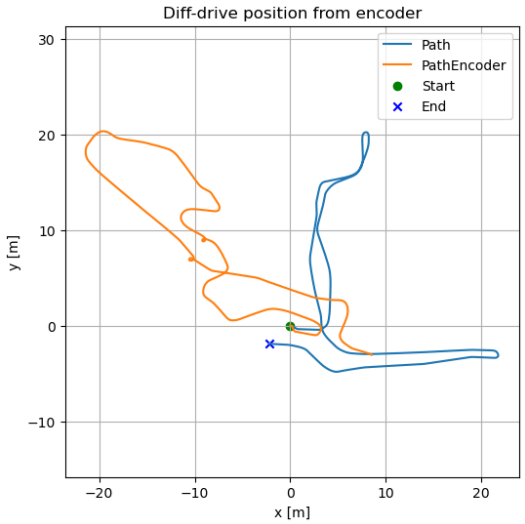
III. Methods and Approach

A. IMU and Encoder Pose Estimation

We estimate the IMU and robot pose using the equations defined below, and use values for wheel diameter, encoder values, and the respective conversion to m/s provided by the datasheet. We manage to get a relatively accurate motion model, since the sensors are not noisy, and the data we are collecting is in a controlled environment.

$$v_r = (FR + RR)/2 * wheel_diameter$$
$$v_l = (FL + RL)/2 * wheel_diameter$$
Our velocity is then the average of v_r and v_l:
$$v = (v_r + v_l)/2$$

After obtaining the velocity and the angular velocity from the IMU and encoders, we can build a simple 2-D motion model to estimate the pose at each timestep.



B. ICP Pose Estimation

The ICP algorithm functions as follows:

ICP Algorithm

m - known LiDAR scan

z - LiDAR scan to estimate relative pose of

$({}_mT_z)_{est}$ - estimated pose from z to m

for n iterations:

$z_{trans} = T * z_{depths}$

$z_{closest} = nearest_points(z_{trans}, m)$

$({}_mT_z)_{new} = kabsch(m, z_{closest})$

$({}_mT_z)_{est} = ({}_mT_z)_{new} * ({}_mT_z)_{est}$

This iteratively moves both point clouds closer till they match, using Kabsch's algorithm to find the relative pose using the closest points between both point clouds.

Kabsch Algorithm

m - known LiDAR scan

z - LiDAR scan to estimate relative pose of (transformed)

$centroid_z$, $centroid_m$: centroid of z and m

$delta_z$, $delta_m$: $z - centroid_z$ and $m - centroid_m$

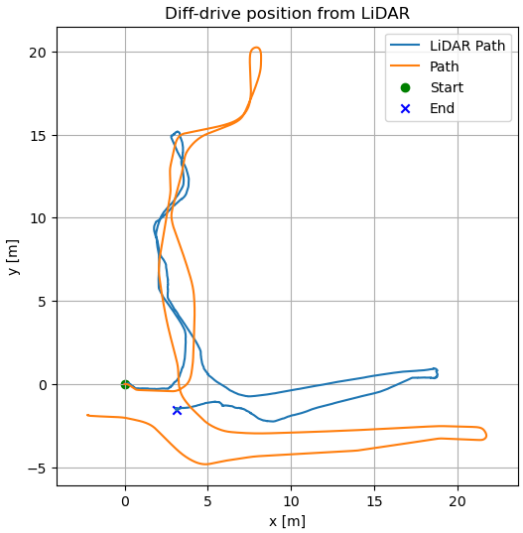
$Q = delta_m * delta_z^T$

$SVD(Q) = U\Sigma V^T$

$${}_mR_z = U* \begin{bmatrix} 1 & 0 & \dots \\ 0 & 1 & \dots \\ \dots & \dots & det(UV^T) \end{bmatrix} *V^T$$

${}_mp_z = centroid_m -_m R_z * centroid_z$

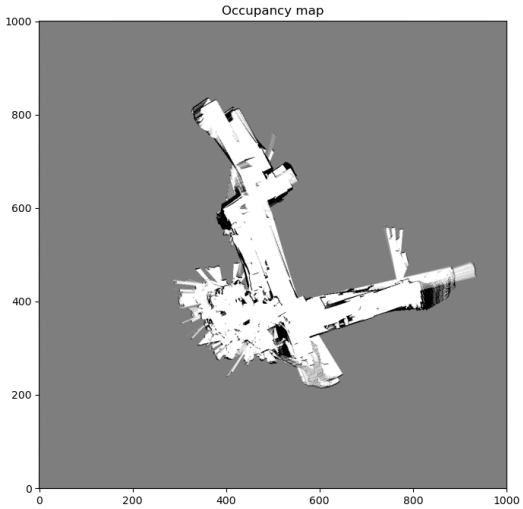
We then get an accurately scaled estimate of the path taken by the robot, albeit with some drift that we hope to improve using loop closure in the next steps.



C. Occupancy and Texture Mapping

Using the LiDAR scans and estimated poses, we can create an Occupancy map. We use the data and the Bresenham2D algorithm to map out the paths each LiDAR ray took in the world, and decrease the log odds of each cell the ray passed through until the endpoint, which we mark as occupied by increasing the log odds. Then, we can map out this 2D array of log odds to reveal an aggregate occupancy map of the entire space.

This map is quite noisy at the start and end points due to the noisy estimate from our LiDAR motion model not being able to converge accurately.



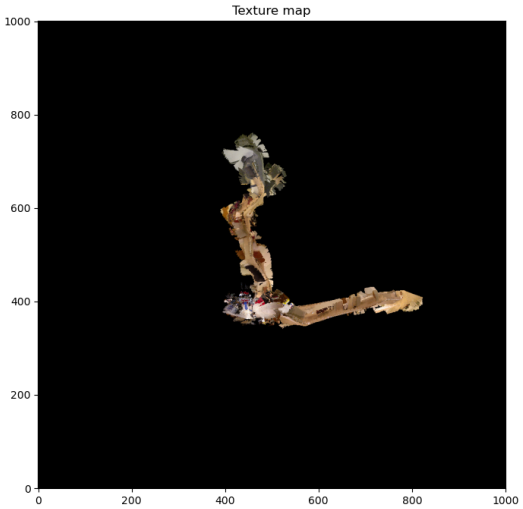
We can also use our RGBD data to map textures to our occupancy map. In a similar method to our occupancy mapping, we assign a color to the ground for all pixels where the depth matches an approximate ground value. Upon doing this for all images and associated poses, we can build an approximate texture map.

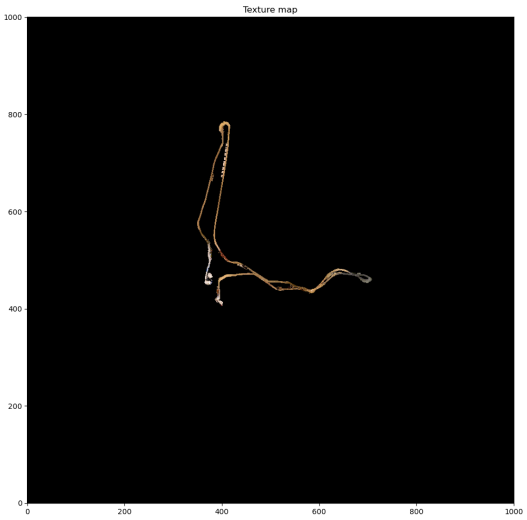
We use the following algorithm to build our texture map:

Texture Mapping Algorithm

for n iterations:

$d = calculate_true_depth(depth)$
$img_i, img_j = calculate_relevant_pixel(d)$
$points_cam = project_image(intrinsics, img_i, img_j, rgb$
$points_world =_{world} T_{robot} *_{robot} T_{cam}$
$if(points_world[z] < 0)$
$\quad texture_map.append(points_world)$





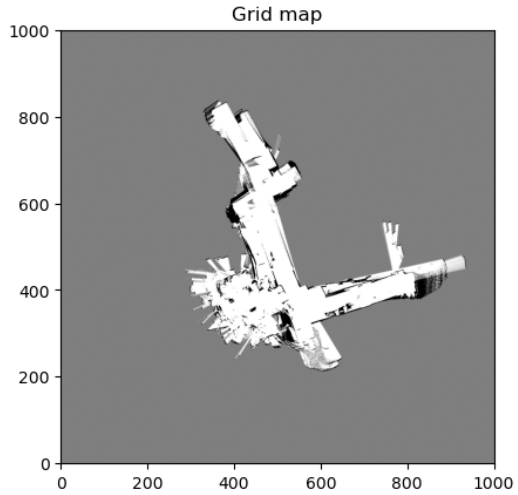
IV. Results

D. Loop Closure and Factor Graph SLAM

To achieve more accurate results, we build a factor graph using the gtsam library, and optimize it using loop closures by proximity and over fixed time intervals.

We take a fixed-time interval of 1 second to attempt loop closure, to ensure near poses are correctly accounted for. For proximity loop closures, we keep an offset of 2 seconds, but check every timestep with every previous measurement taken 2 seconds or more in the past. The threshold we have defined for distance is 0.1m, and for our ICP loss is 10^{-3}.

After adding loop closures, we can then optimize our factor graph mapping using Gauss Newton or Levenberg Marquardt, and then display our optimized Occupancy and Texture Maps.



We show our results on both datasets provided to us, labelled dataset 20 and 21. We will display the results of all stages of optimization: IMU mapping, LiDAR scan matching, Occupancy Mapping, Texture Mapping, and Factor Graph SLAM.

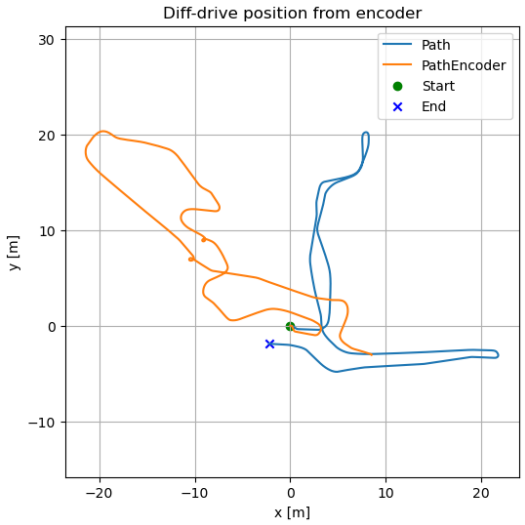
With the occupancy map, we can see that it contains a certain level of noise, which is caused due to the log odds being equivalent for calculations since we do not have noise estimates.

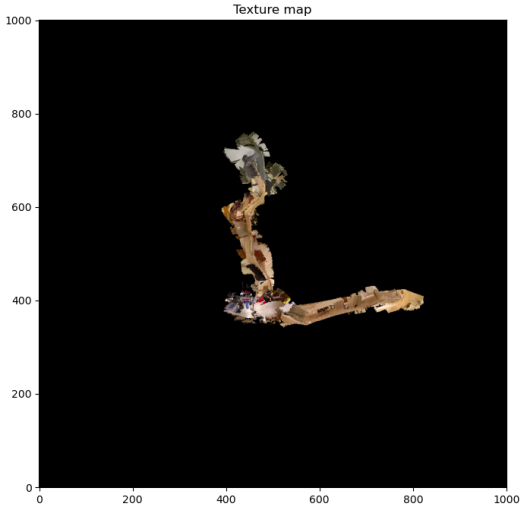
There exist a few drawbacks with the texture map, as can be seen with the images provided. Through the implementation, it appears as though the depth calculation for the images is incorrect, leading to incorrect perceived image depths. We believe that with further work or a different implementation (such as using different hardware), we will be able to move past these specific issues.

Our loop closure using gtsam was also mediocre, so we implemented manual loop closure near the endpoints to ensure accurate results during implementation.

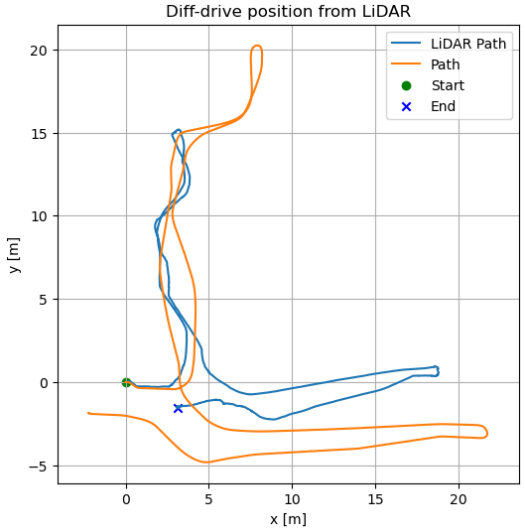
A. Dataset 20

1) IMU Motion:

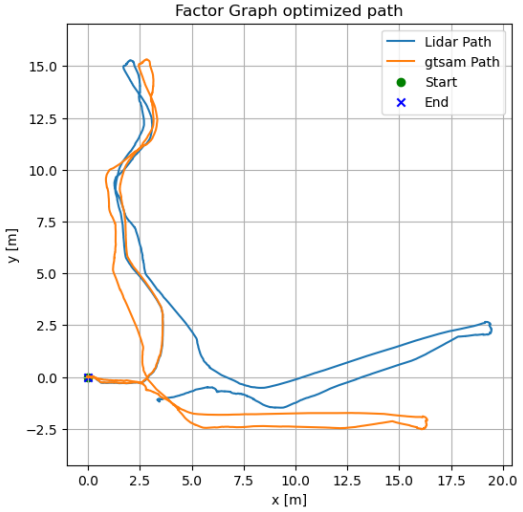




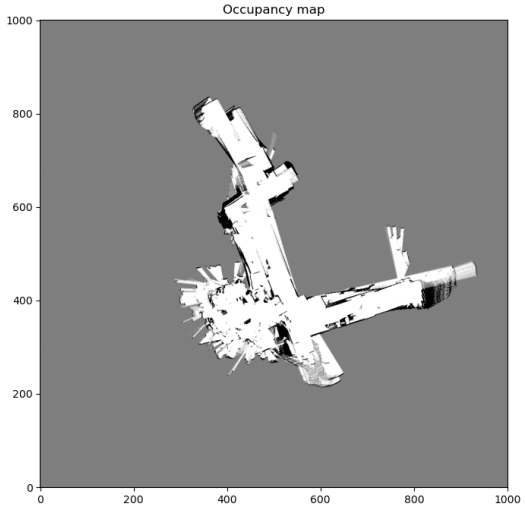
2) LiDAR Scan Matching:



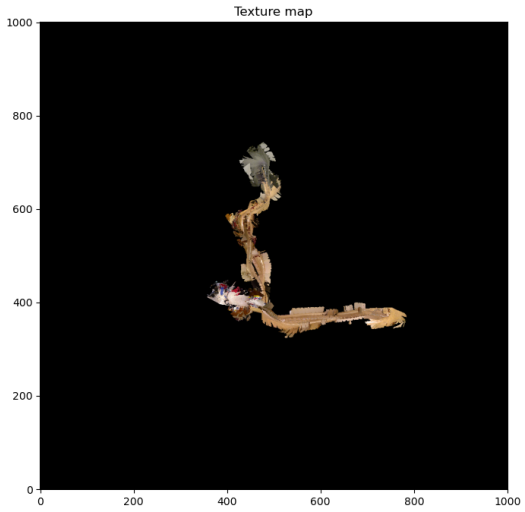
4) Factor Graph SLAM:



3) Occupancy and Texture Mapping:



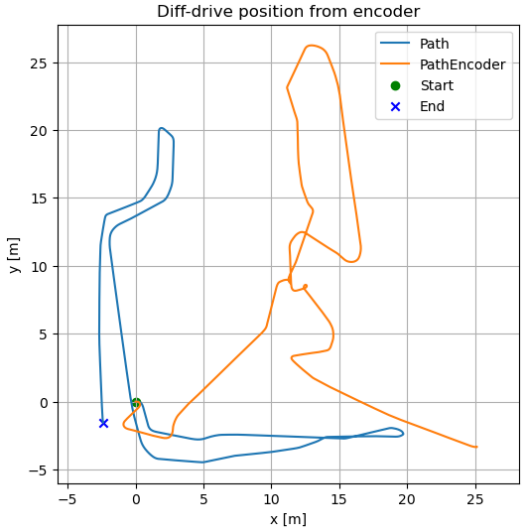




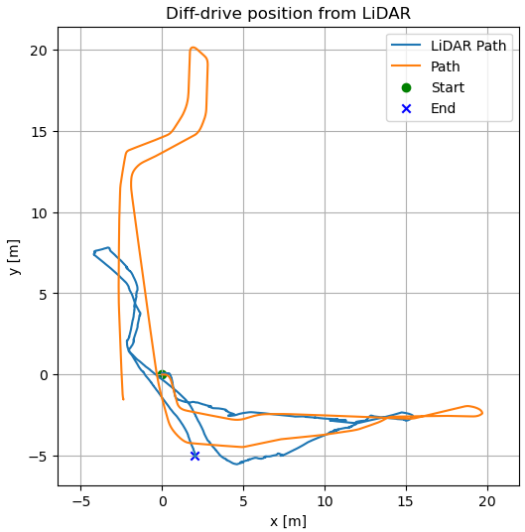


B. *Dataset* 21

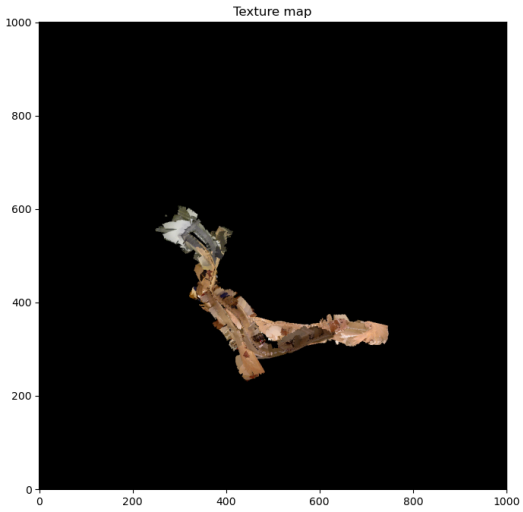
1) *IMU Motion:*



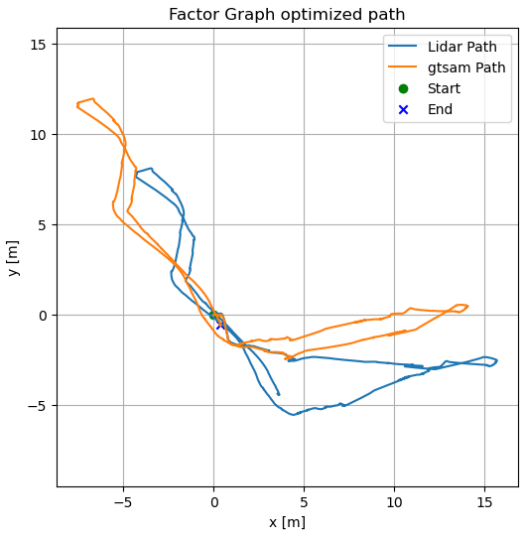
2) *LiDAR Scan Matching:*



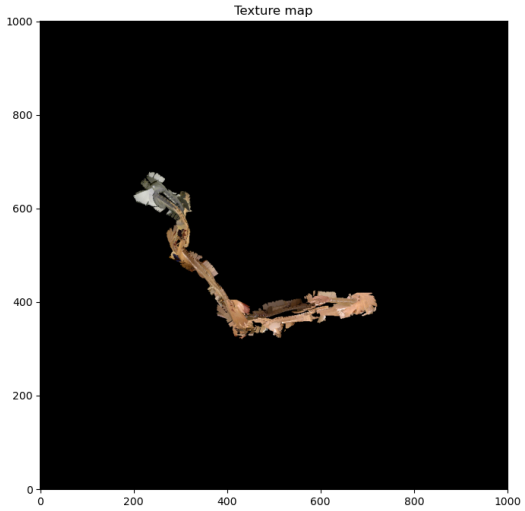
3) *Occupancy and Texture Mapping:*



4) *Factor Graph SLAM:*







V. Conclusion

We present a method of mapping by utilizing LiDAR measurements along with IMU measurements, and implementing new algorithms like ICP and Factor Graphs to optimize our mapping output. We apply this method on two datasets, and tune the parameters to increase the accuracy of our output.

A notable weakness in our results is the lack of an accurate texture map, which is due to an inaccurate depth calculation, as described in Section 4. We will leave it to future work to build an improved texture map to visualize camera data along with occupancy mapping.

References

1) "2.1. Pose2 SLAM — GTSAM by Example." 2021, gtbook.github.io/gtsam-examples/Pose2SLAMExample.html.